

How the Compliance Score Is Generated

A step-by-step walkthrough of how each entry in `compliance_matrix.yaml` is read by `scoring_engine.py` to produce a 0–100 priority score, with four worked examples drawn from real test emails.

4	6	2	18
Scoring steps	YAML categories with weights	Multipliers from YAML	max_raw_score (normalisation ceiling)

1. What the YAML File Contains

All scoring inputs live in **config/compliance_matrix.yaml**. The scoring engine reads this file once at startup and uses it for every calculation. No code changes are needed to tune the scoring — only the YAML.

Category weights

Every category has a **base_weight** (0–10) reflecting severity. This is the primary input to the score formula.

```
categories:

market_manipulation:
base_weight: 10 # Most severe – direct market crime

bribery:
base_weight: 9 # Very severe – improper payments

secrecy:
base_weight: 7 # Severe – MNPI leakage to external parties

employee_ethics:
base_weight: 7 # Serious – conflicts of interest

change_in_communication:
base_weight: 6 # Concerning – evading surveillance

complaints:
base_weight: 6 # Important – regulatory/client grievances
```

Scoring parameters

Multipliers, normalisation ceiling, priority bands, and confidence thresholds are all set here.

```
scoring:

multipliers:
multi_category_bonus: 1.15 # applied when 2+ categories detected
high_confidence_bonus: 1.10 # applied when confidence >= 0.80
external_recipient_bonus: 1.10 # applied when email goes outside org

max_raw_score: 18 # Normalisation ceiling – raw divided by this x 100

per_category_score_cap:
single_category: 90 # max score when only 1 category detected
```

```
two_categories: 100 # no cap when 2+ categories detected

priority_bands:

critical: 85 # score >= 85
high: 65 # score >= 65
medium: 40 # score >= 40
low: 0

confidence_thresholds:

high: 0.80 # >= 0.80 -> Alert: HIGH
medium: 0.55 # >= 0.55 -> Alert: MEDIUM
low: 0.00 # < 0.55 -> Alert: LOW
```

2. The 4-Step Scoring Formula

scoring_engine.py executes these four steps in order for every non-compliant email. All values come directly from the YAML.

1 Sum the base weights from YAML

Read `base_weight` from the YAML for each category the AI detected. Add them together to get the raw score.

2 Apply multi-category bonus (if 2 or more categories)

If the AI detected 2 or more categories, multiply raw by `multipliers.multi_category_bonus` (1.15 from YAML). Multiple violations in one email are more serious than a single one.

3 Apply high-confidence bonus (if confidence ≥ 0.80)

If the AI's confidence is at or above `confidence_thresholds.high` (0.80), multiply by `multipliers.high_confidence_bonus` (1.10). A high-certainty finding is weighted more than a borderline one.

4 Normalise to 0-100 and apply single-category cap

`priority_score = min(100, int(raw / max_raw_score * 100))`. `max_raw_score` is 18 in the YAML. If only one category was detected the score is also capped at `per_category_score_cap.single_category` (90).

The formula in scoring_engine.py

```
# Step 1 – sum weights from YAML
raw = 0.0
for cat_id in categories:
    raw += config['categories'][cat_id]['base_weight']

# Step 2 – multi-category bonus
if len(categories) >= 2:
    raw *= config['scoring']['multipliers']['multi_category_bonus'] # 1.15

# Step 3 – high-confidence bonus
if confidence >= config['scoring']['confidence_thresholds']['high']: # 0.80
    raw *= config['scoring']['multipliers']['high_confidence_bonus'] # 1.10

# Step 4 – normalise and cap
score = min(100, int(raw / config['scoring']['max_raw_score'] * 100))
if len(categories) == 1:
```

```
score = min(score, config['scoring']['per_category_score_cap']['single_category']) #  
90
```

3. Worked Examples

Four real test emails, showing every step with the exact YAML value used at each stage.

Example 1 — Market Manipulation (single category, high confidence)

Email: "coordinate our buy orders around 09:31 we can push the price up 4-5% before the retail crowd notices. Don't put this in the system."

Step	YAML value used	Calculation	Running total
1 Sum weights	market_manipulation: base_weight: 10	raw = 10	raw = 10.0
2 Multi-category	Only 1 category detected	Not applied (need >= 2)	raw = 10.0
3 High-confidence	high_confidence_bonus: 1.10 high threshold: 0.80 AI confidence = 0.96	0.96 >= 0.80 YES raw = 10.0 x 1.10	raw = 11.0
4 Normalise & cap	max_raw_score: 18 single_category cap: 90	int(11.0/18x100) = 61 Apply cap: min(61, 90) = 61	score = 61
Band	high: 65 medium: 40	61 >= 40, < 65 -> HIGH	HIGH
Alert level	high threshold: 0.80 confidence = 0.96	0.96 >= 0.80 -> HIGH alert	Alert: HIGH

61 / 100

Result: priority_score = 61/100 | Band: HIGH | Alert: HIGH (single-category cap prevents CRITICAL — correct for one violation)

Example 2 — Secrecy + Change in Communication (two categories)

Email: "Don't put anything about the Vega deal in email. Use Signal from now on. Delete this thread."

Step	YAML value used	Calculation	Running total
1 Sum weights	secrecy: base_weight: 7 change_in_communication: base_weight: 6	raw = 7 + 6	raw = 13.0
2 Multi-category	multi_category_bonus: 1.15 (2 categories found)	2 >= 2 YES raw = 13.0 x 1.15	raw = 14.95
3 High-confidence	high_confidence_bonus: 1.10 high threshold: 0.80 AI confidence = 0.91	0.91 >= 0.80 YES raw = 14.95 x 1.10	raw = 16.45
4 Normalise	max_raw_score: 18 two_categories cap: 100	int(16.45/18x100) = 91 Two-cat — no cap applies	score = 91

Band	critical: 85	91 >= 85 -> CRITICAL	CRITICAL
Alert level	high threshold: 0.80 confidence = 0.91	0.91 >= 0.80 -> HIGH alert	Alert: HIGH

91 / 100

Result: priority_score = 91/100 | Band: CRITICAL | Alert: HIGH (multi-category bonus pushed score from 72 to 91 — appropriate for two violations)

Example 3 — Regulatory Complaint (single category, medium confidence)

Email: "I am formally complaining about three unauthorised trades totalling £240,000. If unresolved I will escalate to the FCA."

Step	YAML value used	Calculation	Running total
1 Sum weights	complaints: base_weight: 6	raw = 6	raw = 6.0
2 Multi-category	Only 1 category detected	Not applied	raw = 6.0
3 High-confidence	high threshold: 0.80 AI confidence = 0.72	0.72 < 0.80 NO Not applied	raw = 6.0
4 Normalise & cap	max_raw_score: 18 single_category cap: 90	int(6.0/18x100) = 33 min(33, 90) = 33	score = 33
Band	medium: 40 low: 0	33 >= 0, < 40 -> LOW	LOW
Alert level	medium threshold: 0.55 confidence = 0.72	0.72 >= 0.55 -> MEDIUM alert	Alert: MEDIUM

33 / 100

Result: priority_score = 33/100 | Band: LOW | Alert: MEDIUM (flagged for review — complaints weight=6 ensures it is never marked COMPLIANT)

Example 4 — Three Categories (bribery + secrecy + change in comms)

Email: "I have non-public ECB guidance. Coordinate EUR/USD before the statement. Keep this off the books — use the Malta entity. Erase any messages."

Step	YAML value used	Calculation	Running total
1 Sum weights	market_manipulation: 10 secrecy: 7 change_in_communication: 6	raw = 10 + 7 + 6	raw = 23.0
2 Multi-category	multi_category_bonus: 1.15 (3 categories found)	3 >= 2 YES raw = 23.0 x 1.15	raw = 26.45
3 High-confidence	high_confidence_bonus: 1.10 high threshold: 0.80 AI confidence = 0.97	0.97 >= 0.80 YES raw = 26.45 x 1.10	raw = 29.10
4 Normalise	max_raw_score: 18 two_categories cap: 100	int(29.10/18x100) = 161 min(161, 100) = 100	score = 100
Band	critical: 85	100 >= 85 -> CRITICAL	CRITICAL
Alert level	high threshold: 0.80 confidence = 0.97	0.97 >= 0.80 -> HIGH alert	Alert: HIGH

100 / 100

Result: priority_score = 100/100 | Band: CRITICAL | Alert: HIGH (three serious categories push raw above the normalisation ceiling — correctly CRITICAL)

Example 5 — Compliant Email (zero score, no calculation)

Email: "Please find attached the Q1 settlement reconciliation. All figures signed off by the risk desk. Deadline Friday 17:00."

The AI returns `is_compliant = true` and `categories = []`.
scoring_engine.py short-circuits immediately — no YAML weights are read, no formula is executed:

priority_score = 0 | band = COMPLIANT | alert = NONE

4. Complete Category Weight Reference

All six categories from the YAML, showing the score produced when each is detected alone at high confidence (0.96), and the resulting band.

Category ID	Label	Weight	Score alone (conf 0.96, cap 90)	Band
market_manipulation	Market Manipulation	10	$\text{int}(10 \times 1.10 / 18 \times 100) = 61 \rightarrow \min(61, 90) = 61$	HIGH
bribery	Market Bribery	9	$\text{int}(9 \times 1.10 / 18 \times 100) = 55 \rightarrow \min(55, 90) = 55$	MEDIUM
secrecy	Secrecy / MNPI	7	$\text{int}(7 \times 1.10 / 18 \times 100) = 42 \rightarrow \min(42, 90) = 42$	MEDIUM
employee_ethics	Employee Ethics	7	$\text{int}(7 \times 1.10 / 18 \times 100) = 42 \rightarrow \min(42, 90) = 42$	MEDIUM
change_in_communication	Change in Comms	6	$\text{int}(6 \times 1.10 / 18 \times 100) = 36 \rightarrow \min(36, 90) = 36$	LOW
complaints	Complaints	6	$\text{int}(6 \times 1.10 / 18 \times 100) = 36 \rightarrow \min(36, 90) = 36$	LOW

* Single-category cap (90) applies in all rows above. Two-category combinations bypass the cap and can reach 100.

5. How to Tune the Scoring

Every parameter is in the YAML. Changes take effect on the next run — no Python code changes required.

To achieve this...	Edit this YAML entry	Effect
Raise market_manipulation priority	<code>market_manipulation:</code> <code>base_weight: 10 -> 10</code> (already max)	Already highest — combine with multi-category for 100
Make complaints flag as HIGH	<code>complaints: base_weight: 6 -> 8</code>	Single complaint alone scores 48 (MEDIUM); with bonus 57 (MEDIUM->HIGH boundary)
Make CRITICAL threshold tighter	<code>priority_bands: critical: 85 -> 90</code>	Only the most severe multi-category findings reach CRITICAL
Require higher confidence for HIGH alert	<code>confidence_thresholds: high: 0.80 -> 0.90</code>	Only very certain findings trigger HIGH alert; borderline ones become MEDIUM
Reduce multi-category boost	<code>multi_category_bonus: 1.15 -> 1.05</code>	Two-category combinations get 5% boost instead of 15%

Add a new violation
category

```
categories: insider_trading:  
  base_weight: 10 label: ...  
  keywords: [...]
```

New category immediately available — also add to
SYSTEM_PROMPT in compliance_agent.py

Note: The **out_of_scope_categories** and **canonical_aliases** sections in the YAML are configuration data — but the actual remapping logic must also be referenced in the AI system prompt (**compliance_agent.py** → **SYSTEM_PROMPT**) so the model knows which category IDs are valid and how to map aliases. Changing a weight or threshold requires only the YAML; adding a new category requires both the YAML and the system prompt.